

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Oberst Eduard, FAF-243

Report

Laboratory work nr. 1

The OS as a Resource Manager

Course: Operating Systems (Track A: Linux)

Checked by:
Patricia Reitman, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Environment

The lab was run on native Ubuntu (dual boot with Windows), not on the provided VM, so the numbers below come from real laptop hardware.

```
$ whoami
capziro
$ uname -a
Linux capziro-3 7.0.0-31-generic #31-Ubuntu SMP PREEMPT_DYNAMIC
Sat Aug  1 04:26:38 UTC 2026 x86_64 GNU/Linux
$ uptime
 15:48:34 up  1:22,  1 user,  load average: 1.77, 1.69, 1.29
```

The machine is managed by Linux kernel 7.0.0 on a 64-bit (x86_64) processor, and the OS had been running for 1 hour and 22 minutes.

Part 1. Files and directories (File management)

Interesting command output:

```
$ ls -l note.txt
-rw-rw-r-- 1 capziro capziro 24 Oct  5 15:51 note.txt
$ chmod 600 note.txt
$ ls -l note.txt
-rw------- 1 capziro capziro 24 Oct  5 15:51 note.txt
```

Observations:

1. **Who owns the files you just created, and which group?**
   The files are owned by my user `capziro` and belong to the group `capziro`. Ubuntu creates a personal group with the same name for every user. By default the new file got `rw-rw-r--`, which matches my umask of `0002`.

2. **What do the ten characters at the start of `ls -l` mean for note.txt after chmod 600?**
   After `chmod 600` the string is `-rw-------`:

   - Character 1 (`-`): it is a regular file (a directory would show `d`, a link `l`).
   - Characters 2–4 (`rw-`): the owner can read and write, but not execute.
   - Characters 5–7 (`---`): the group has no rights.
   - Characters 8–10 (`---`): all other users have no rights.

   In octal, 6 = 4 (read) + 2 (write) and 0 = nothing, so only I can access the file.

3. **Name two directories directly under / and say what they hold.**

   - `/etc`: system-wide configuration files (for example `NetworkManager`, `X11`, `adduser.conf`).
   - `/home`: the personal folders of users. Mine is `/home/capziro`.

   I also noticed `/swap.img` (4 294 967 296 bytes = 4 GiB) directly under `/`. It is the swap file I found again in Part 3.

Part 2. Processes (Process management)

Interesting command output:

```
$ top
top - 15:56:04 up  1:30,  1 user,  load average: 1.44, 1.26, 1.20
Tasks: 391 total,   1 running, 389 sleeping,   0 stopped,   1 zombie
%Cpu(s):  6.1 us,  2.0 sy,  0.0 ni, 91.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  15181.1 total,   5448.3 free,   5762.6 used,   5578.6 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   9418.5 avail Mem

$ cat /proc/$PID/status | head -20
Name:   sleep
State:  S (sleeping)
Pid:    27430
PPid:   22991
Uid:    1000    1000    1000    1000
VmPeak:    16112 kB
VmSize:    16112 kB
```

Observations:

1. **What is the PID of process number 1, and what is it?**
   `ps -p 1` shows that PID 1 is `systemd` (`/usr/lib/systemd/systemd`). It is the first user-space process that the kernel starts at boot. It starts all system services, and every other user process descends from it.

2. **Roughly how many processes were running?**
   `ps aux | wc -l` returned 393 lines. Minus the header line, that is about 392 processes. `top` reported 391 tasks: 1 running, 389 sleeping and 1 zombie. So almost all processes were waiting, not using the CPU, which matches the 91.8% idle CPU.

3. **What does the "State:" line say for your sleeping process?**
   It says `State:  S (sleeping)`. The `sleep 300` process is in interruptible sleep: it waits for its timer and uses no CPU until then. `PPid:  22991` shows that its parent is my bash shell.

Part 3. Memory (Memory management)

Interesting command output:

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:            14Gi       5.6Gi       5.3Gi       1.6Gi       5.4Gi       9.2Gi
Swap:          4.0Gi          0B       4.0Gi
$ swapon --show
NAME      TYPE SIZE USED PRIO
/swap.img file   4G   0B   -1
$ grep VmRSS /proc/$PID/status
VmRSS:      7716 kB
```

Observations:

1. **How much total RAM is there, and how much is free?**
   `/proc/meminfo` shows `MemTotal: 15545472 kB`, about 14.8 GiB (a 16 GB laptop; part of the memory is reserved for hardware such as integrated graphics). About 5.3 GiB was completely free, but 9.2 GiB was *available*. The difference is the 5.4 GiB of buffers/cache, which Linux uses to speed up disk access and can give back to programs when they need it.

2. **What is swap, and how much is configured?**
   Swap is space on the disk that the OS uses as an extension of RAM. When RAM is full, rarely used memory pages are moved there. On my system, swap is a 4 GiB file, `/swap.img`, and 0 B was in use, because there was still plenty of free RAM.

3. **How much resident memory (VmRSS) does a bare sleep process use?**
   `VmRSS` was 7716 kB (about 7.5 MB), while `VmSize` was 16112 kB. It surprised me a little that a program which does nothing uses several megabytes of real RAM. The reason is that even `sleep` must load the C library (`libc`), the dynamic loader and its stack. Many of those pages are shared with other processes, so the real extra cost is smaller. `VmSize` is bigger than `VmRSS` because not all of the virtual address space is actually loaded into RAM.

Part 4. Devices and storage (I/O management)

Interesting command output:

```
$ lsblk   (snap loop devices omitted)
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 953.9G  0 disk
|-nvme0n1p1 259:1    0   100M  0 part /boot/efi
|-nvme0n1p2 259:2    0    16M  0 part
|-nvme0n1p3 259:3    0 487.4G  0 part
|-nvme0n1p4 259:4    0   887M  0 part
‘-nvme0n1p5 259:5    0 464.7G  0 part /
$ findmnt /
TARGET SOURCE         FSTYPE OPTIONS
/      /dev/nvme0n1p5 ext4   rw,relatime
```

Observations:

1. **Which device is your root filesystem "/" mounted on?**
   `/` is mounted on `/dev/nvme0n1p5`, the 5th partition of my NVMe SSD. It is formatted as ext4 (457 GB, 7% used). The same disk also holds the EFI partition (`p1`, mounted on `/boot/efi`) and partitions `p2`–`p4`, which are not mounted in Ubuntu. Those belong to Windows (`p3` is the 487 GB Windows system partition), so one physical disk is shared by two operating systems.

2. **Give one entry from /dev and say what real thing it stands for.**
   `/dev/nvme0n1` stands for the whole physical NVMe SSD. Another example: `/dev/console` (a character device, `c`, numbers 5,1) is the system console terminal. I also saw that `lsblk` lists many `loop` devices: each snap application (Firefox, Telegram, VS Code) is a file that the OS mounts as if it were a separate disk.

3. **In one sentence: what does "everything is a file" mean?**
   It means that Linux shows disks (`/dev/nvme0n1`), terminals (`/dev/console`), running processes (`/proc/<pid>/`) and memory information (`/proc/meminfo`) as files in one directory tree, so programs can read and use all of them with the same simple file operations.

Conclusion

In this lab I saw the operating system working as a manager of four resources on my own Ubuntu laptop. For **files**, `ls -l` and `chmod` showed how the OS controls who owns a file and who may read or write it. For **processes**, `ps aux` and `top` showed about 390 processes, almost all sleeping, all descending from `systemd` (PID 1). For **memory**, `free -h` showed how 15 GB of RAM is divided between programs and cache, with a 4 GiB swap file kept in reserve. For **devices**, `lsblk` and `df -h` showed the NVMe disk, its partitions (including the Windows ones), and where each filesystem is mounted. The `/proc` and `/dev` directories clearly showed the UNIX idea that everything is a file.